BÀI 09: AUTHENTICATION VỚI JWT VÀ PASSPORT TRONG NESTJS

Mục tiêu bài 09: Bài này sẽ giới thiệu đến các bạn về Authentication (xác thực) với JWT và Passport trong Nestjs. Giới thiệu đến các bạn từng bước để tạo xác thực với JWT và Passport trong ứng dụng Nestjs như thế nào.

9.1. TỔNG QUAN VỀ AUTHENTICATION VÀ JWT

Authentication là quá trình xác thực người dùng để cấp quyền truy cập vào hệ thống. *Ví dụ:* Khi bạn đăng nhập vào Gmail bằng tài khoản Google, hệ thống sẽ kiểm tra **username** và **password** có đúng không → đó là **authentication**.

JWT (**Json Web Token**) là một tiêu chuẩn mở cho phép truyền thông tin an toàn giữa các bên dưới dạng **token**. **JWT** thường được sử dụng để xác thực người dùng trong các ứng dụng **RESTful API**.

Cấu trúc **JWT** gồm 3 phần: **<Header>.<Payload>.<Signature>**

Thành phần	Nội dung
Header	Thông tin thuật toán ký (ví dụ: HS256)
Payload	Dữ liệu người dùng (username, role, ...)
Signature	Kết quả mã hóa Header + Payload bằng secret key

Ví dụ kết quả sau khi mã hóa password: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9

.eyJ1c2VybmFtZSI6ImFkbWluIiwiaWQiOjEsImlhdCI6MTY5NzY4ODQ0OH0

.ZnhA3CQXWl9u7n5LNjzUlD-3tZPscA2L7a9djYdSYRQ.

Cách hoạt động của Authentication với JWT:

1. Người dùng gửi yêu cầu đăng nhập (username/password).

2. Server kiểm tra thông tin → Nếu hợp lệ → tạo JWT.

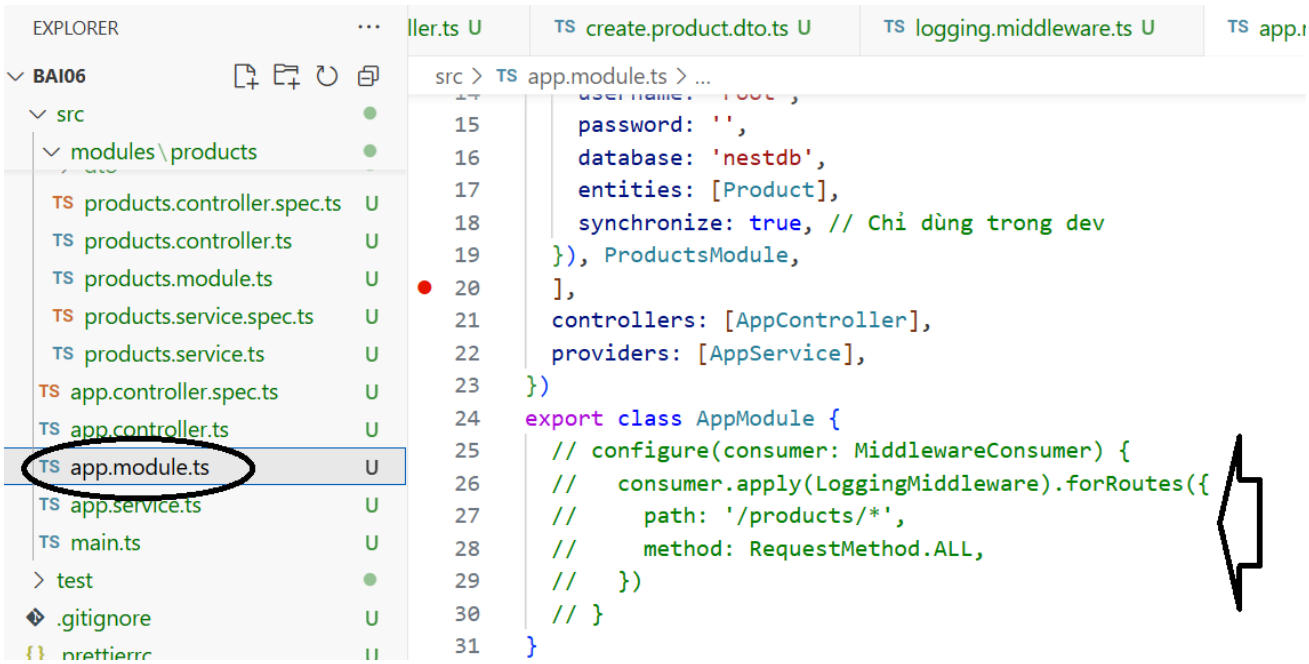
3. JWT được gửi về client (thường lưu ở localStorage hoặc cookie).

4. Client gửi JWT trong header Authorization mỗi lần gọi API.

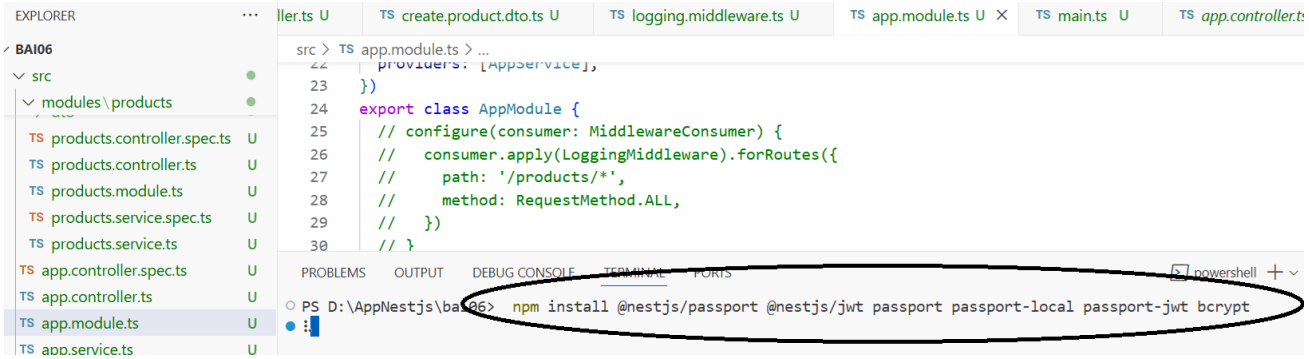
5. Server kiểm tra JWT → Nếu hợp lệ → Cho phép truy cập tài nguyên.

9.2. CÀI ĐẶT CÁC THƯ VIỆN CẦN THIẾT

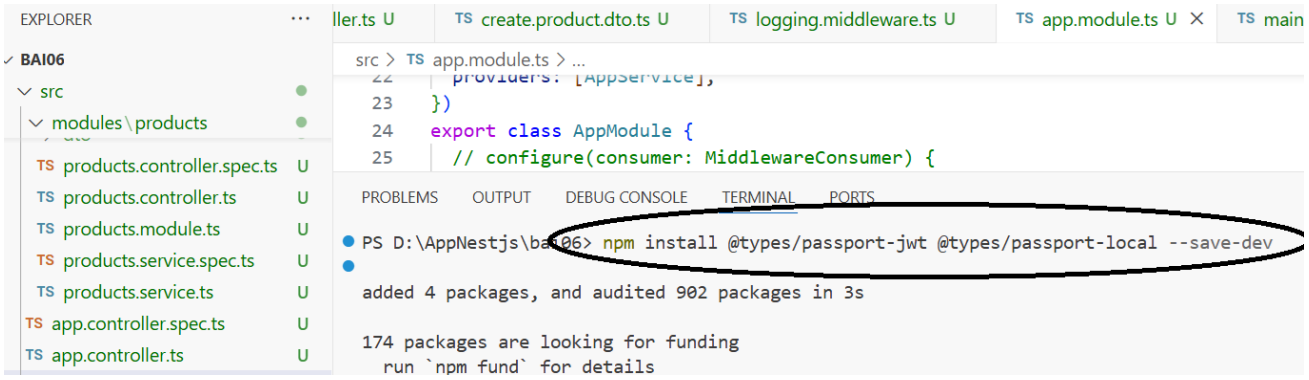
- Các bạn mở lại dự án bài trước, mở tập tin **app.module.ts** và bỏ phần **Middleware** đã làm ở bài trước nhé.



- Mở **Terminal** và thực hiện lệnh để cài đặt thư viện cần thiết: **npm install @nestjs/passport @nestjs/jwt passport passport-local passport-jwt bcrypt**

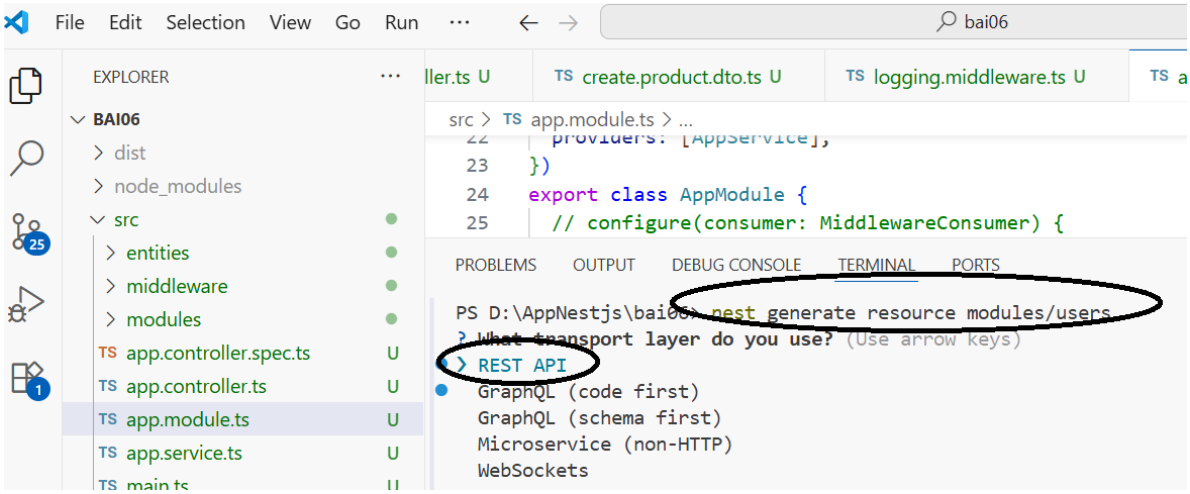


- Tiếp tục cài thư viện **types** với dòng lệnh: **npm install @types/passport-jwt @types/passport-local --save-dev**

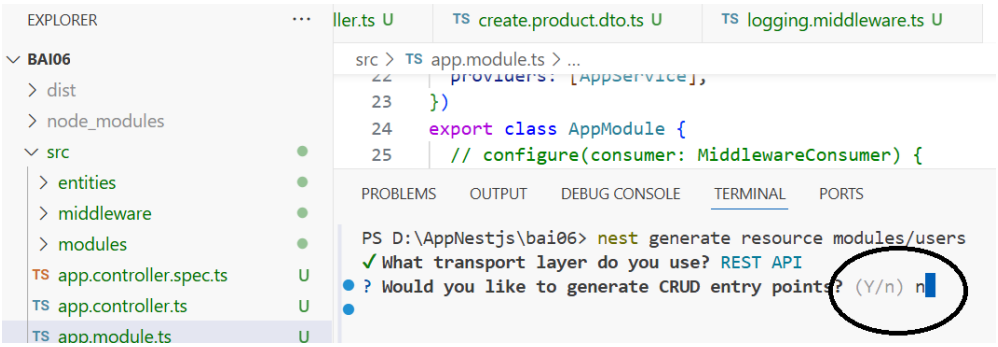


9.3. TẠO MODULE USER

- Tạo thêm Module **users** bằng dòng lệnh: **nest generate resource modules/users**. Chọn **REST API**, bấm **Enter**

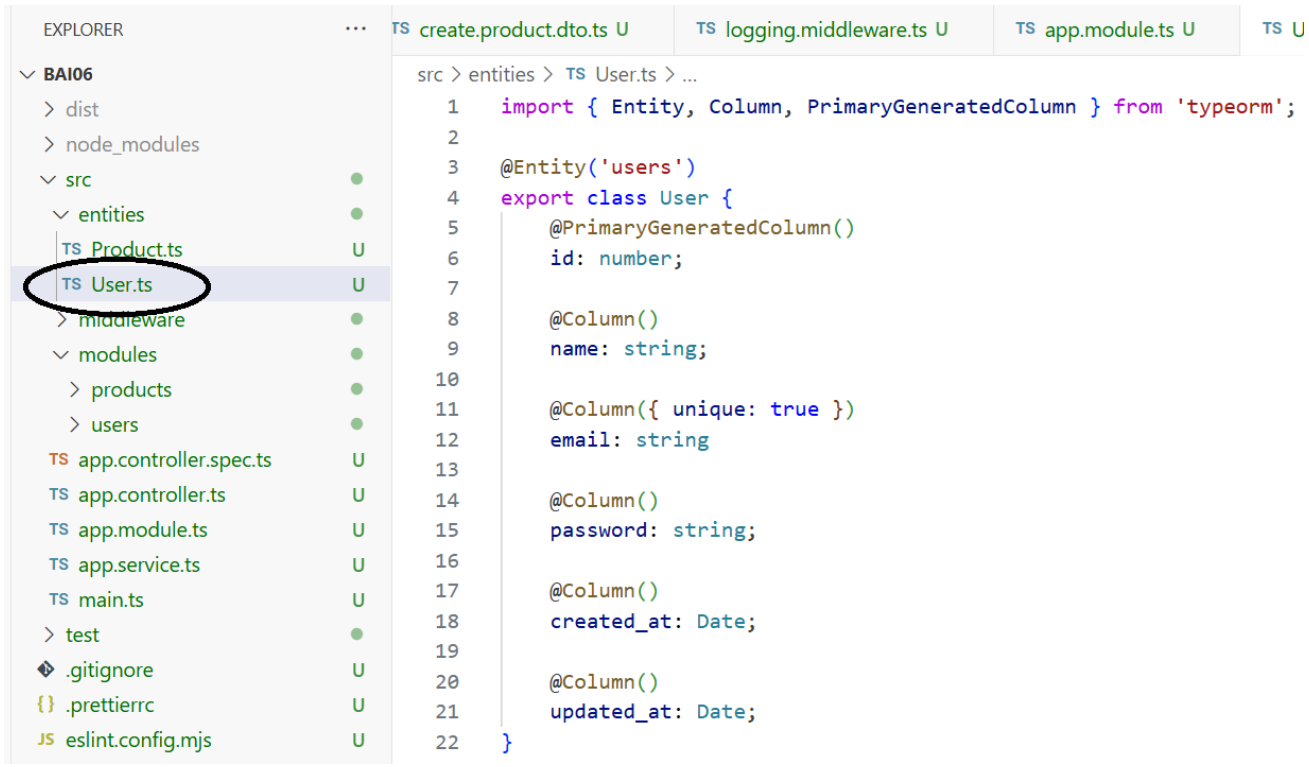


- Chọn **n**, bấm **Enter**, sẽ tạo được **module users**

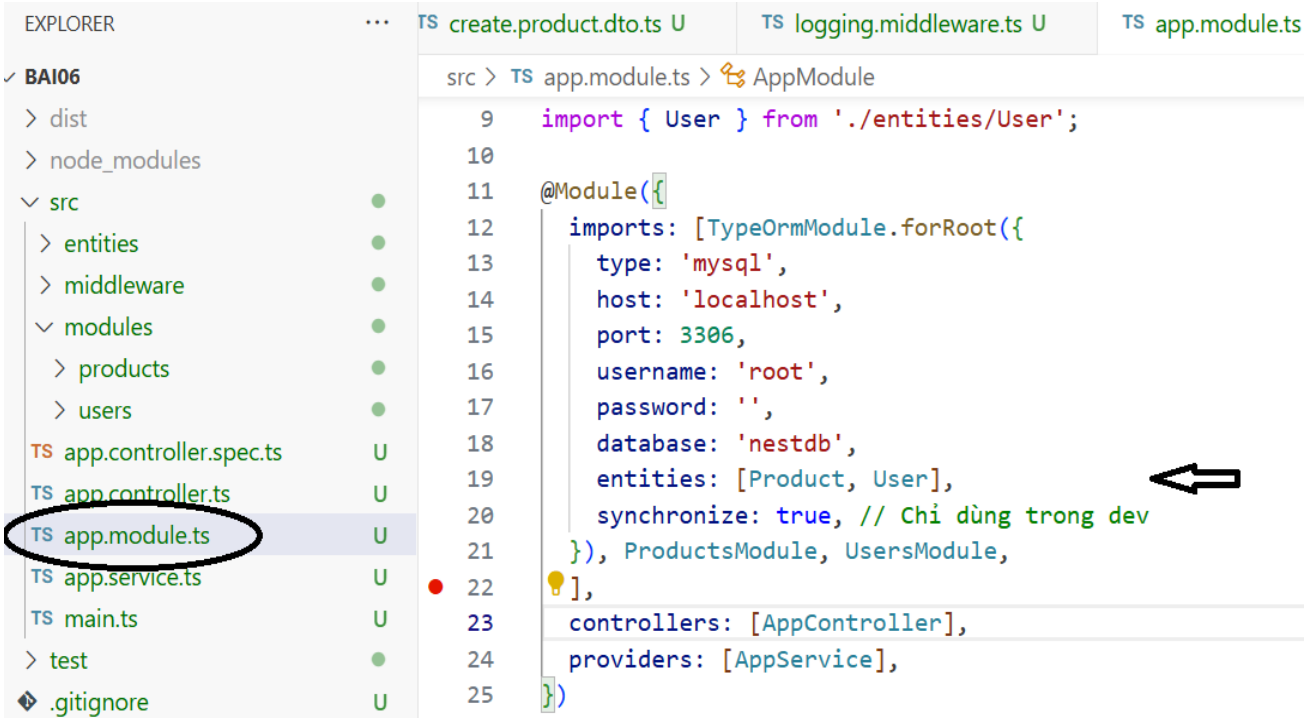


9.4. TẠO ENTITY USER

- Click chuột phải vào thư mục **entities** tạo tập tin **User.ts** và viết code như bên dưới:

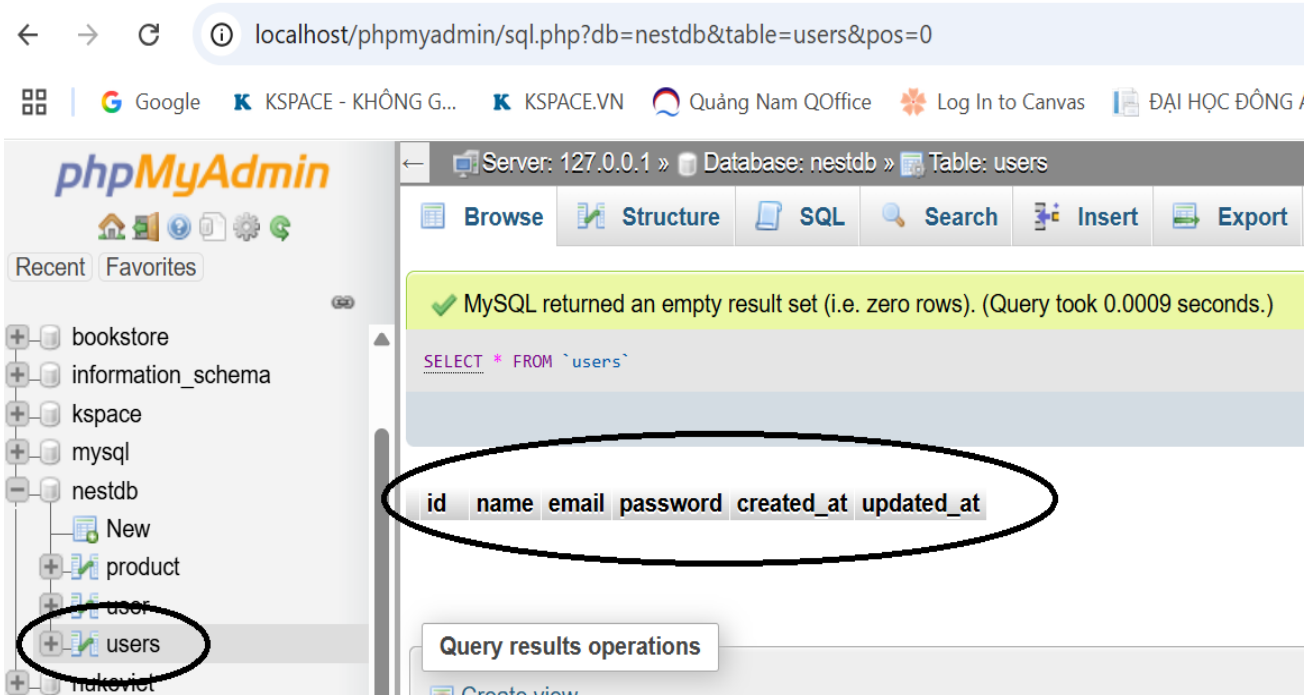


- Tiếp tục gọi **Entity User** vừa tạo vào **Module** bằng cách mở tập tin **app.module.ts** và viết code thêm vào như bên dưới:



- Biên dịch lại chương trình: **npm run start:dev**

- Mở cơ sở dữ liệu lên sẽ thấy tạo thêm 1 bảng **users** và các trường phù hợp

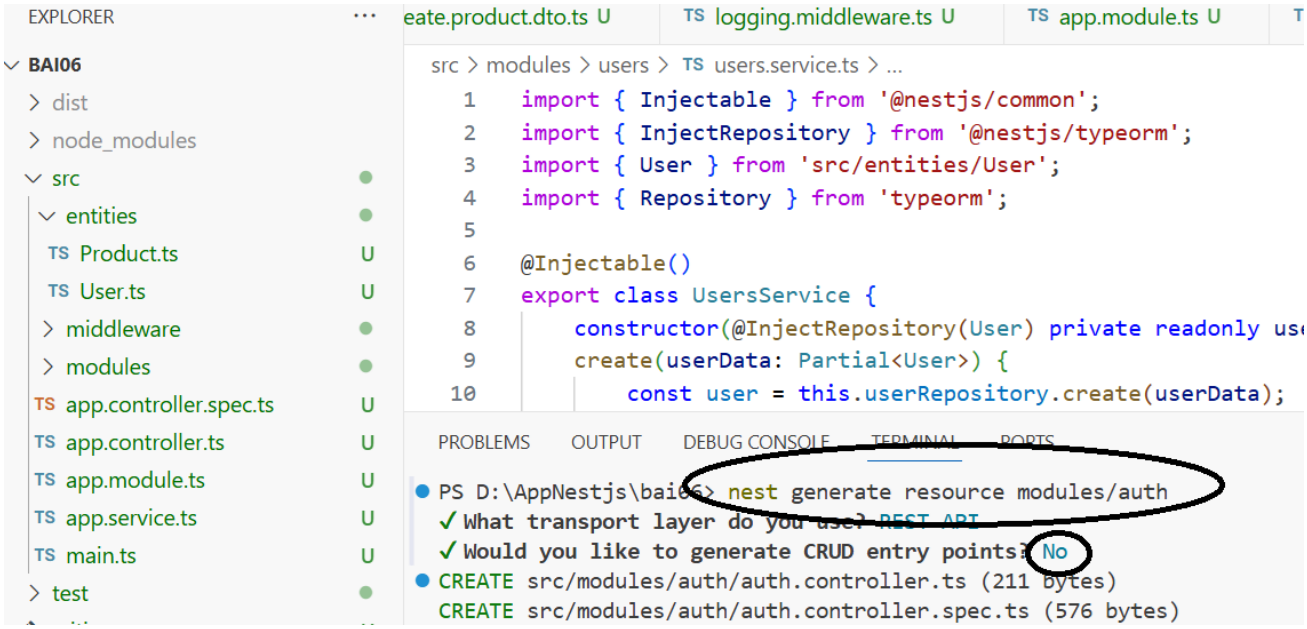


9.5. TẠO CÁC SERVICE ĐỂ QUẢN LÝ USER

- Các bạn mở tập tin **users.service.ts** và viết code để tạo các **Service** như bên dưới:



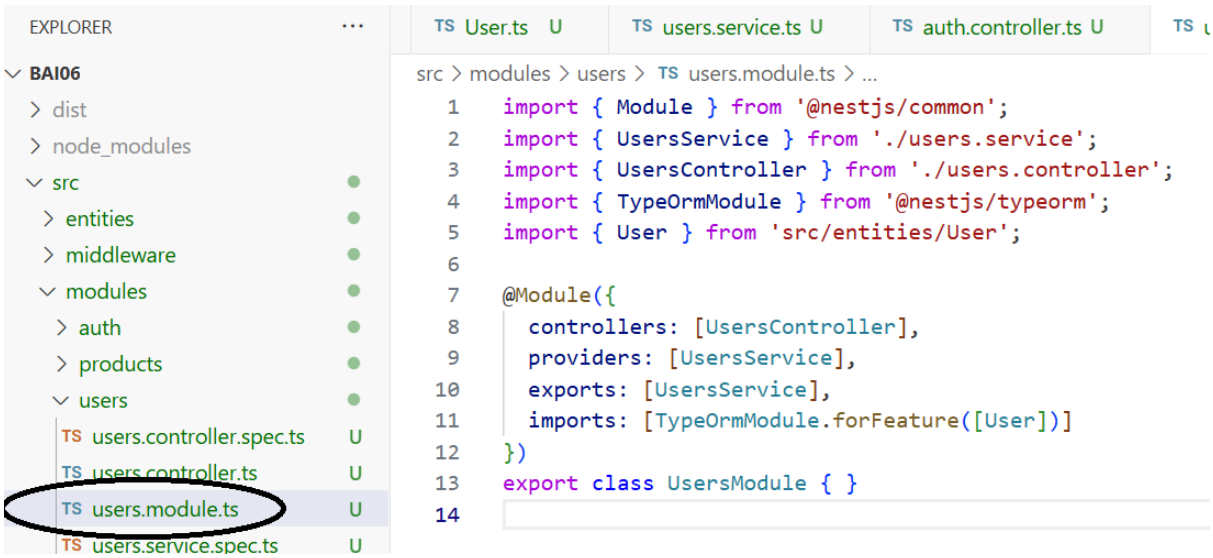
- Tiếp tục mở Terminal để tạo thêm module auth bằng dòng lệnh: **nest generate resource modules/auth**. Chọn **REST API**, bấm **Enter**



- Tiếp tục mở tập tin **auth.controller.ts** và viết code như bên dưới:



- Tiếp tục mở tập tin **users.module.ts** thực hiện **exports** cho các module khác dùng bằng cách viết dòng lệnh như bên dưới:

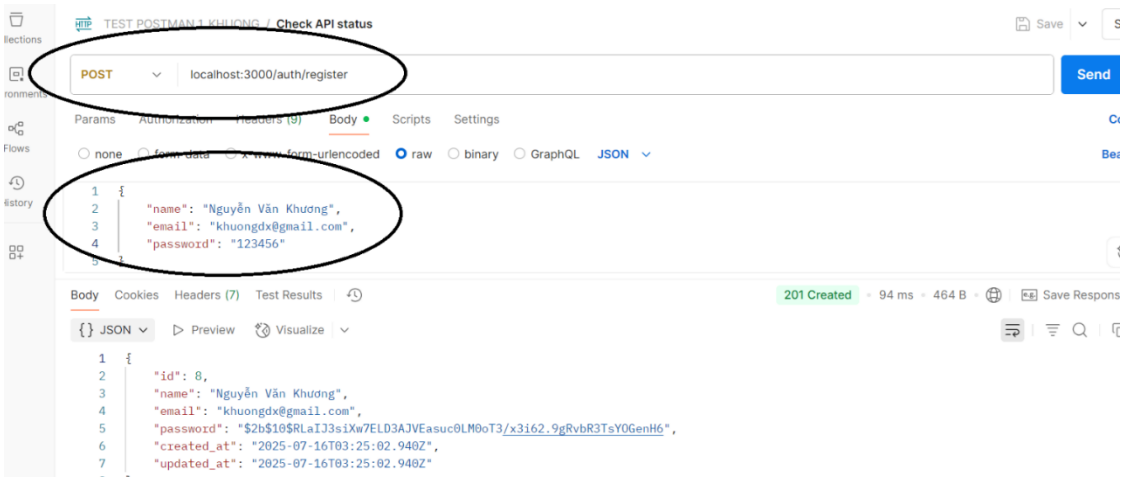


- Tiếp tục mở tập tin **auth.module.ts** thực hiện **imports** Service vào sử dụng như bên dưới:

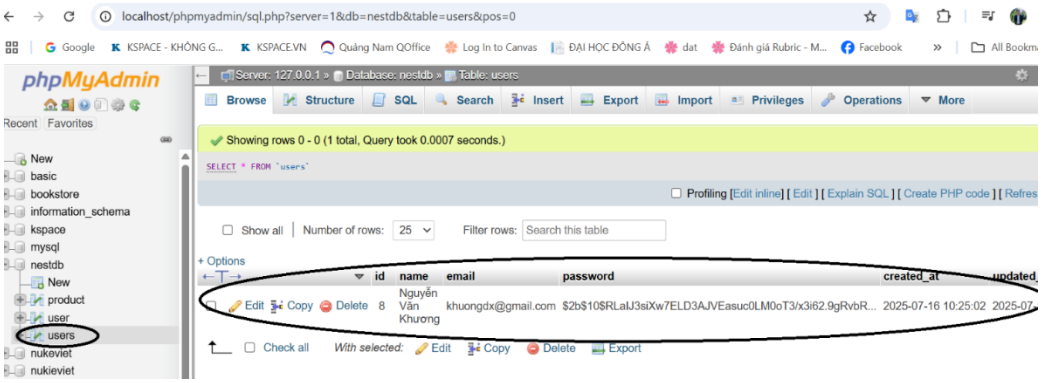


- Biên dịch chương trình bằng dòng lệnh: **npm run start:dev**

- Mở chương trình **Postman** kiểm tra thử, chọn phương thức **POST**, gõ đường dẫn: **localhost:3000/auth/register**, gõ nội dung dữ liệu cần thêm vào trong **raw** và chọn **Send** sẽ thấy dữ liệu được đưa vào cơ sở dữ liệu **MySQL**.

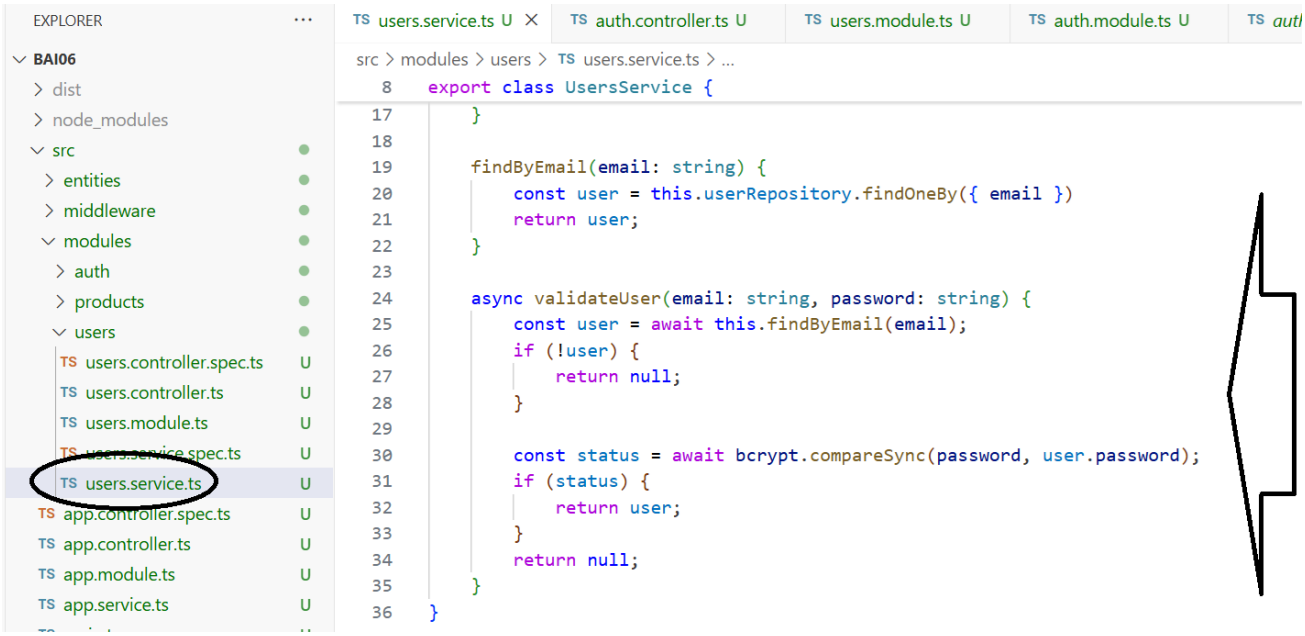


- Mở cơ sở dữ liệu **users** sẽ thấy kết quả:

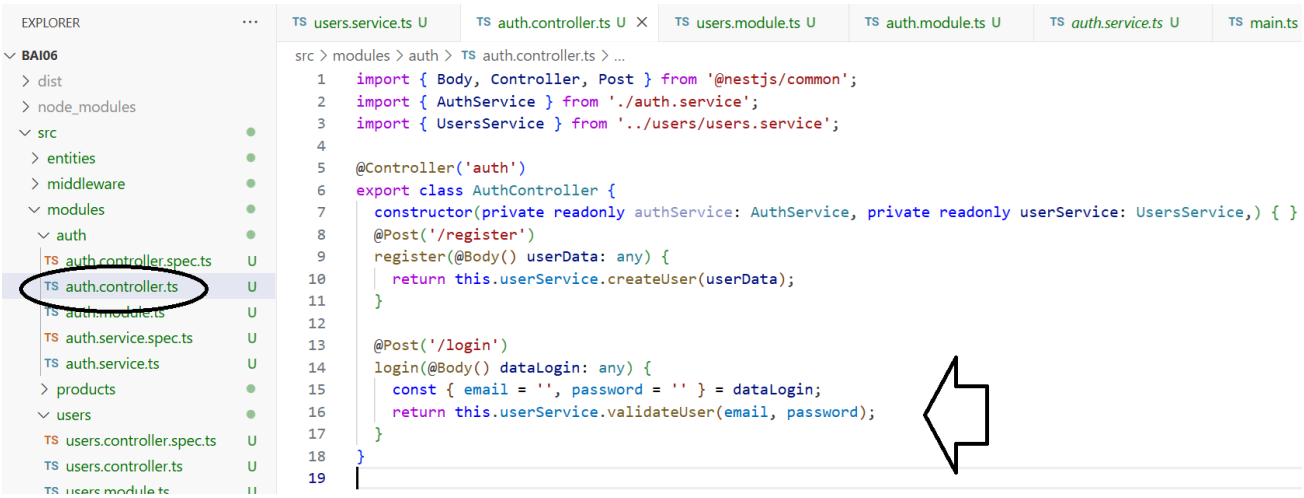


- Như vậy chúng ta đã thực hiện xong phần đăng kí người dùng cho ứng dụng

- Tiếp tục chúng ta viết thêm **Service** nữa (2 service là: **findByEmail ()** và **validateUser ()** cho tập tin **users.service.ts** như bên dưới:

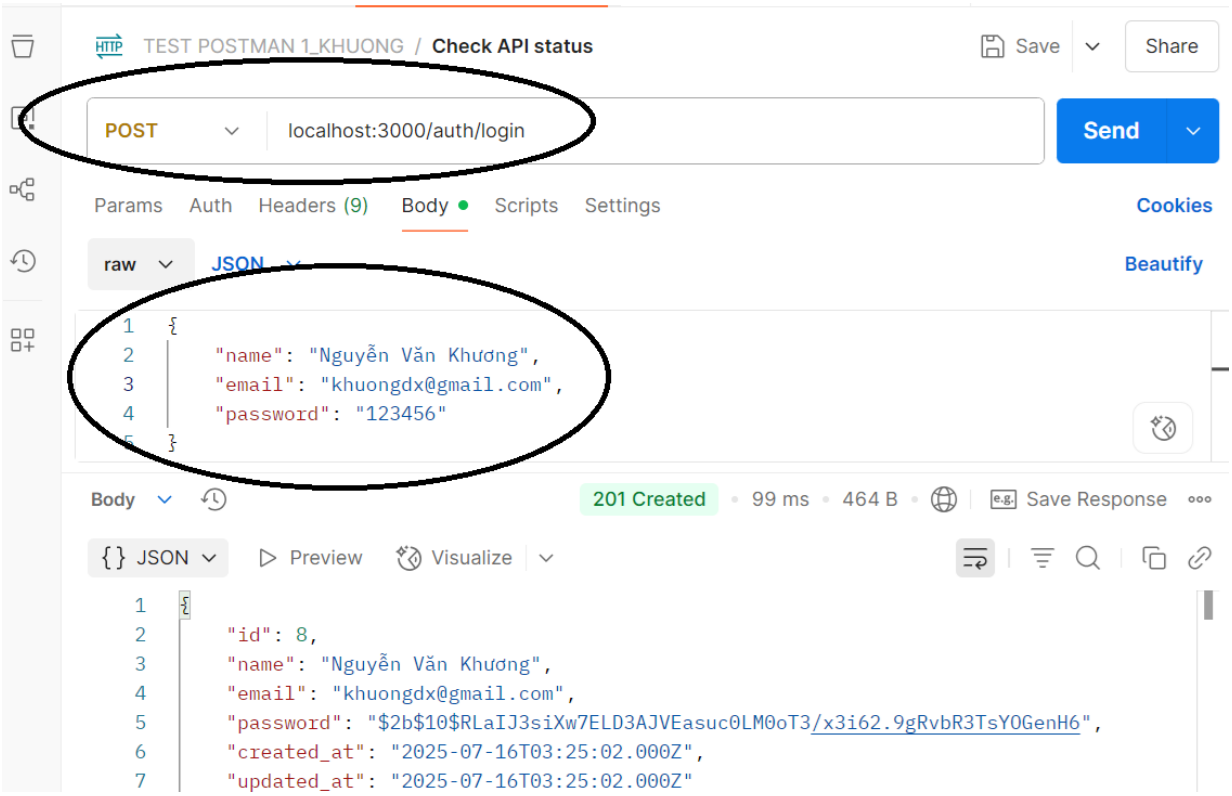


- Tiếp tục mở tập tin **auth.controller.ts** để đưa **service** vào **controller** như bên dưới:



- Biên dịch chương trình: **npm run start:dev**

- Mở chương trình **Postman**, chọn phương thức **POST**, gõ đường dẫn: **localhost:3000/auth/login** sẽ có kết quả như bên dưới:



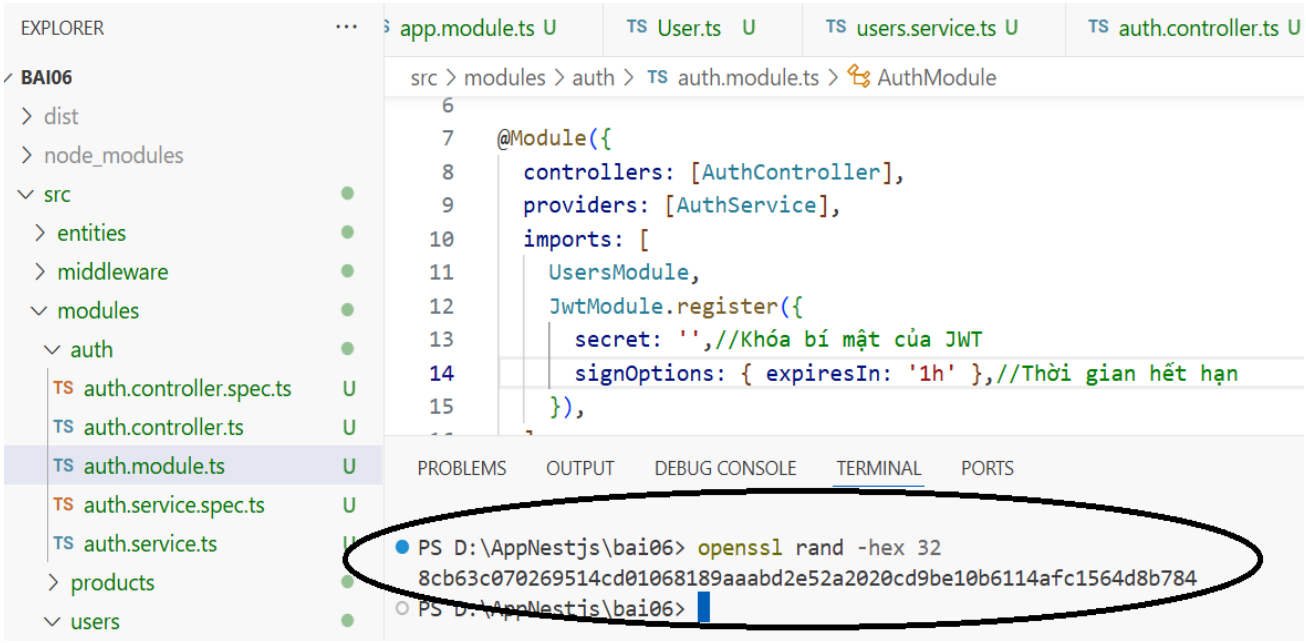
Như vậy ta thấy, sau khi login với đúng Email là: "*khuongdx@gmail.com*", mã hóa mật khẩu: "*123456*" so sánh trùng với dữ liệu mã hóa mật khẩu trong cơ sở dữ liệu thì cho đăng nhập.

9.6. CẤU HÌNH JWT VÀ PASSPORT

- Để cấu hình **JWT** thì các bạn mở tập tin **auth.module.ts** và viết code như bên dưới:



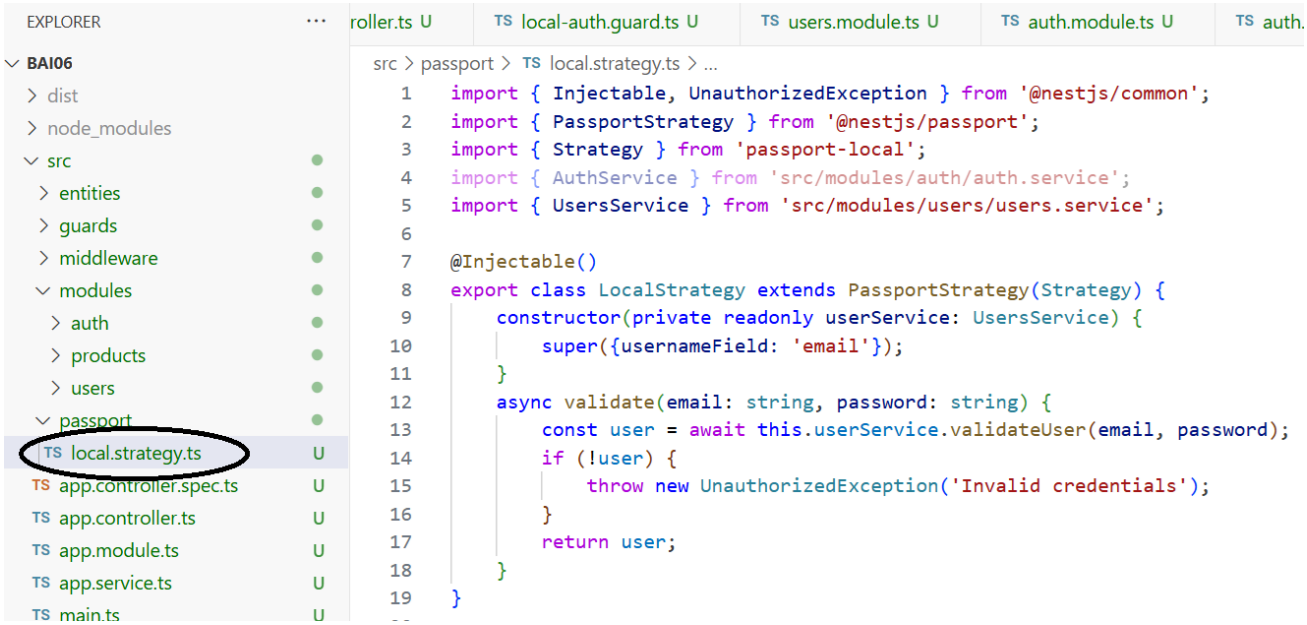
- ***Lưu ý:*** Để có mã số bí mật, các bạn có thể mở **Terminal**, gõ dòng lệnh: **openssl rand -hex 32**, xong copy mã số đó đưa vào **auth.module.ts** (Nếu khi gõ lệnh **openssl rand -hex 32** mà không chạy được thì xem hướng dẫn ở trang **kspace.vn**) *(Mã này nên được lưu vào biến môi trường ở bài học sau chứ không nên lưu vào đây nhé)*



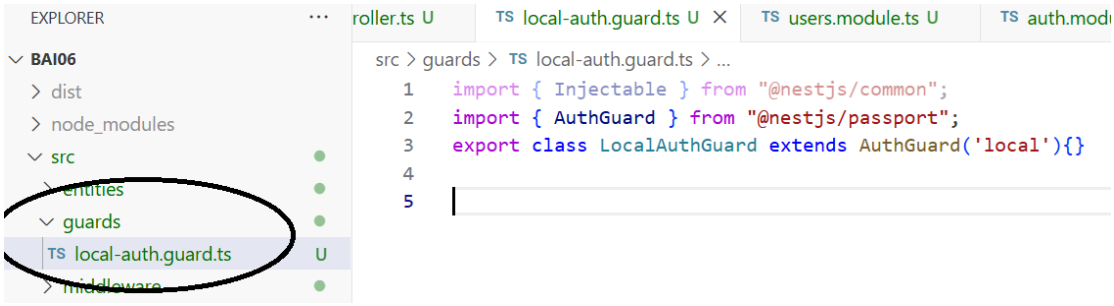
- Tiếp tục, các bạn tạo các **AuthService** để cấp **Token** bằng cách: mở tập tin **auth.service.ts** và viết code như sau:



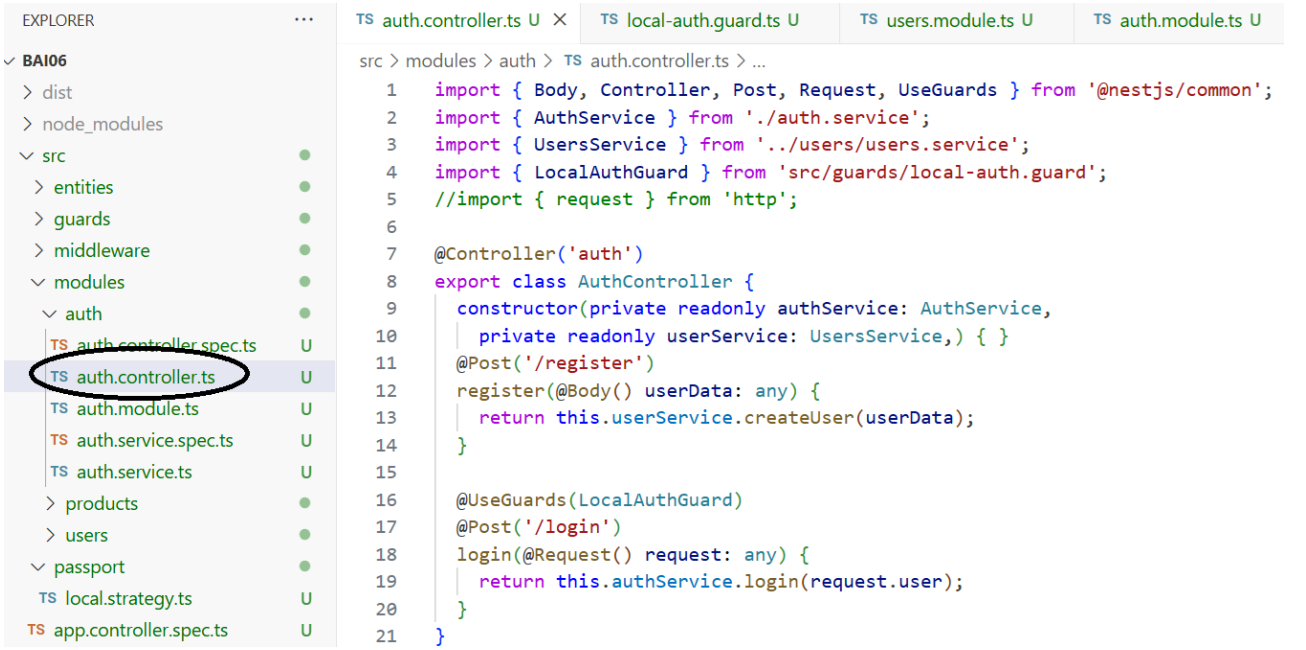
- Tiếp tục, các bạn sẽ tạo **Passport** (một **middleware xác thực** - authentication middleware) bằng cách tạo một thư mục **passport** trong thư mục **src**. Sau đó tạo một tập tin **local.strategy.ts** trong thư mục **passport** và viết code như bên dưới:



- Tiếp theo chúng ta sẽ thực hiện bảo vệ **Authentication** bằng cách tạo thêm 1 thư mục **guards** trong thư mục **src** và tập tin tin **local-auth.guard.ts** trong thư mục **guards** này và viết code như sau:

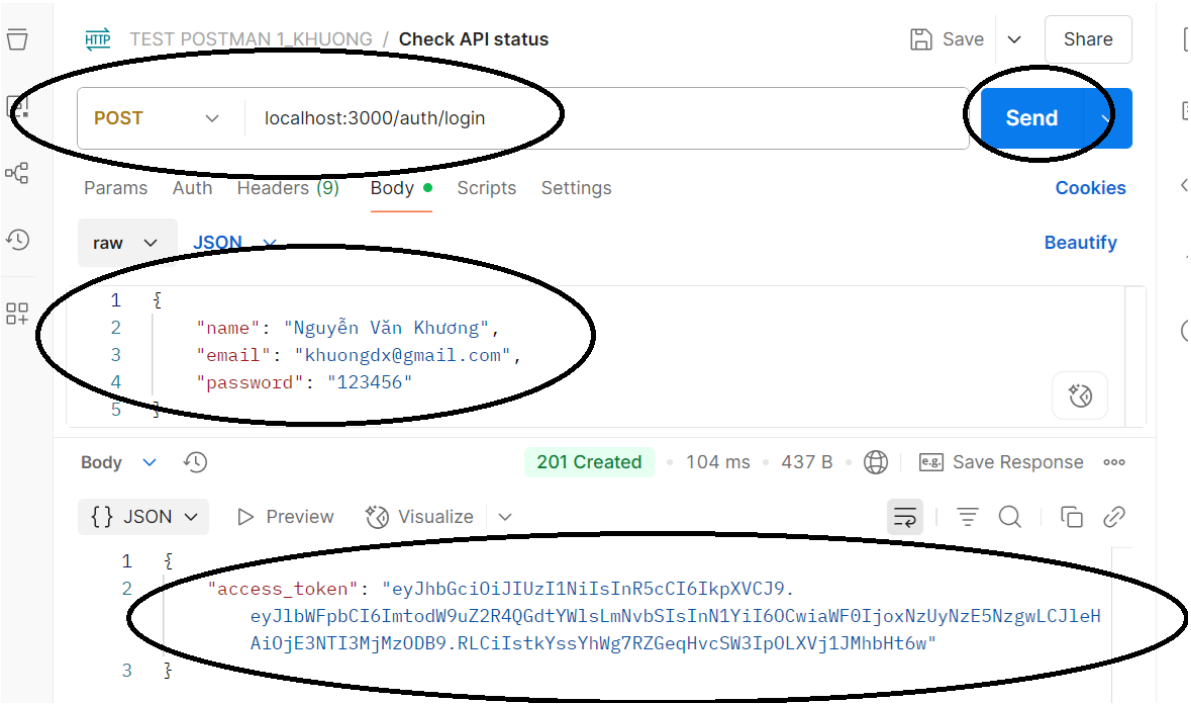


- Tiếp tục, các bạn tạo **Controller** cho **Authentication** bằng cách mở lại tập tin **auth.controller.ts** và sửa lại phương thức **login ()** như bên dưới:



- Mở Terminal biên dịch chương trình: **npm run start:dev**

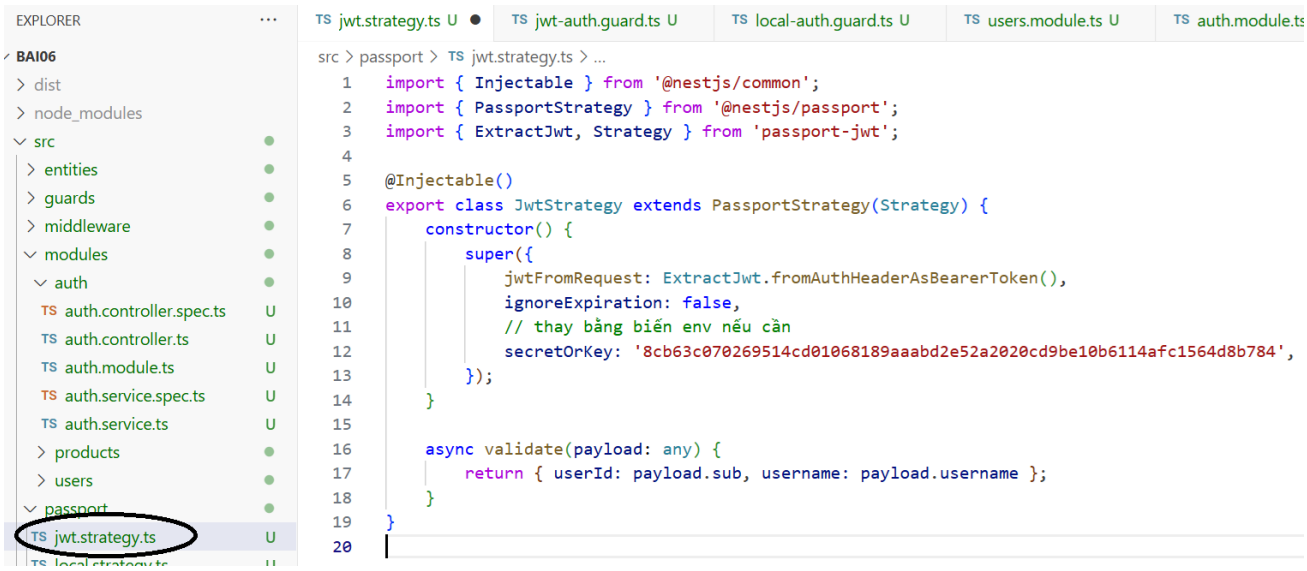
- Mở chương trình **Postman** kiểm tra, chọn phương thức **POST**, gõ đường dẫn: **localhost:3000/auth/login**, gõ dữ liệu đăng nhập vào **raw**, chọn **Send** sẽ cho kết quả: **token_access** với mã hóa bên dưới:



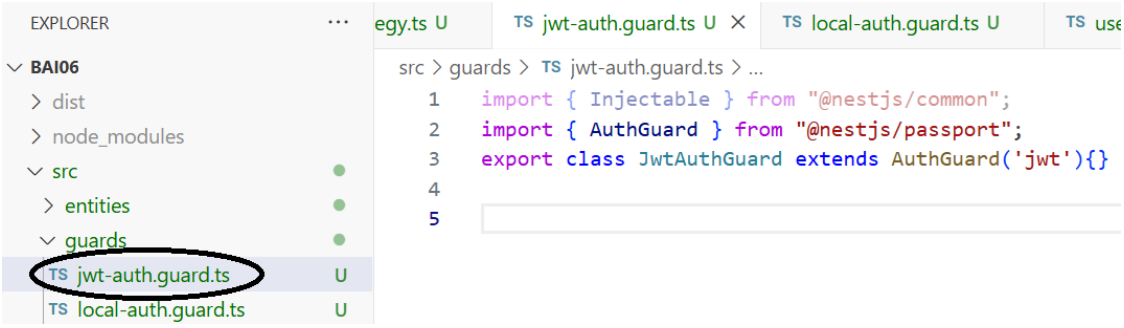
- Như vậy qua chương trình ta thấy: khi gõ đường dẫn **login** thì thông qua **Guards** sẽ gọi **LocalAuthGuard**, khi đó sẽ sử dụng **Passport** trả về thông tin **user**, thông tin này sẽ được đưa vào **Request**, sau đó sẽ gọi sang **AuthService** để lưu dữ liệu đăng nhập mã hóa vào **Token**.

9.7. TẠO JWT STRATEGY ĐỂ BẢO VỆ ROUTE

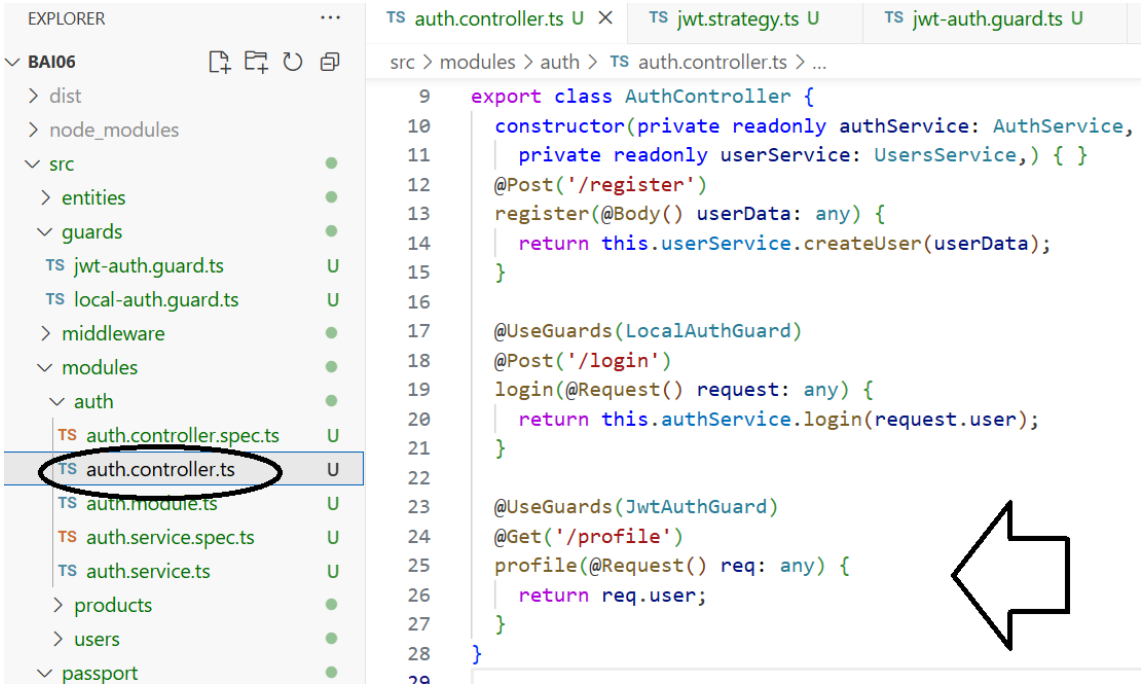
- Tiếp tục, các bạn tạo trong thư mục **passport** tập tin **jwt.strategy.ts** và viết code như bên dưới. Mã **secretOrKey** lấy từ tập tin **auth.module.ts**



- Tiếp tục, các bạn vào thư mục **guards** tạo thêm tập tin **jwt-auth.guard.ts** và viết code như bên dưới.



- Tiếp tục mở tập tin **auth.controller.ts** và viết thêm code như bên dưới *(Nếu lỗi thì import các thư viện cần vào)*



- Tiếp tục các bạn mở tập tin **auth.module.ts** thực hiện **import** vào **module** bằng cách sửa dòng lệnh sau, thêm **JwStrategy**:



- Mở Terminal biên dịch chương trình: **npm run start:dev**

- Mở chương trình **Postman** kiểm tra, mở sang một **tab** mới bằng cách bấm vào dấu +

- Thiết lập cho **tab** mới, chọn phương thức **GET**, gõ đường dẫn: **localhost:3000/auth/profile**, chọn **Authorization**, chọn **AuthType**: **Bearer Token**, copy mã **Token access** ở **tab** bên trái và dán vào mục **Token**. Bấm **Send**, sẽ thấy xuất hiện **userId** đã được xác thực.

